

Set 1:

- Create a 3×3 array of all True Boolean values
- Extract odd numbers from an array
- Replace all odd numbers with -1
- Reverse columns of a 2D array
- Get indices of elements greater than 5 [hint: check where method of numpy]
- Access last row - last column

```
In [2]: import numpy as np
```

- Create a 3×3 array of all True Boolean values

```
In [3]: true_array = np.full((3,3), True, dtype=bool)
true_array
```

```
Out[3]: array([[ True,  True,  True],
               [ True,  True,  True],
               [ True,  True,  True]])
```

- Extract odd numbers from an array

```
In [4]: arr = np.random.randint(1,50,9).reshape(3,3)
arr
```

```
Out[4]: array([[ 8, 22,  4],
               [41, 28,  9],
               [23, 30, 34]], dtype=int32)
```

```
In [5]: odd_numbers = arr[arr % 2 != 0]
odd_numbers
```

```
Out[5]: array([41,  9, 23], dtype=int32)
```

- Replace all odd numbers with -1

```
In [6]: arr[arr % 2 != 0] = -1
arr
```

```
Out[6]: array([[ 8, 22,  4],
               [-1, 28, -1],
               [-1, 30, 34]], dtype=int32)
```

- Reverse columns of a 2D array

```
In [7]: arr1 = np.random.randint(1,20,6).reshape(2,3)
arr1
```

```
Out[7]: array([[14, 11,  3],
               [ 6, 11, 11]], dtype=int32)
```

```
In [8]: np.fliplr(arr1)
```

```
Out[8]: array([[ 3, 11, 14],
               [11, 11,  6]], dtype=int32)
```

- Get indices of elements greater than 5

```
In [9]: arr = np.random.randint(1,20,9).reshape(3,3)
arr
```

```
Out[9]: array([[ 4,  3,  9],
               [18,  3, 14],
               [14,  4, 16]], dtype=int32)
```

```
In [10]: get_indices = np.argwhere(arr > 5)
get_indices
```

```
Out[10]: array([[0, 2],
                [1, 0],
                [1, 2],
                [2, 0],
                [2, 2]])
```

- Access last row - last column

```
In [11]: arr[2,2]
```

```
Out[11]: np.int32(16)
```

Set 2:

- Normalize a NumPy array [to normalize means scaling the values of an array into a specific range, often [0, 1]]
- Create a 5×5 array with 1 on border and 0 inside
- Create a random array of 10×10: perform stats (min, max, mean, std)
- Broadcast a 1D array over a 2D array [all possible scenarios]

```
In [12]: arr = np.random.randint(1,20,9).reshape(3,3)
arr
```

```
Out[12]: array([[17, 12, 19],
               [ 5, 16,  9],
               [ 3,  7, 14]], dtype=int32)
```

- Normalize a NumPy array [to normalize means scaling the values of an array into a specific range, often [0, 1]]

```
In [13]: normalized = (arr - np.min(arr))/(np.max(arr) - np.min(arr))
normalized
```

```
Out[13]: array([[0.875 , 0.5625, 1.    ],
               [0.125 , 0.8125, 0.375 ],
               [0.    , 0.25  , 0.6875]])
```

- Create a 5×5 array with 1 on border and 0 inside

```
In [14]: arr = np.ones((5,5),dtype=int)
arr
```

```
Out[14]: array([[1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1]])
```

```
In [15]: arr [1:4,1:4] = 0
arr
```

```
Out[15]: array([[1, 1, 1, 1, 1],
                [1, 0, 0, 0, 1],
                [1, 0, 0, 0, 1],
                [1, 0, 0, 0, 1],
                [1, 1, 1, 1, 1]])
```

- Create a random array of 10×10: perform stats (min, max, mean, std)

```
In [16]: arr = np.random.randint(1,500,100).reshape(10,10)
arr
```

```
Out[16]: array([[469, 390, 317, 264,  54, 141, 114, 473, 403,  94],
                [ 97, 325, 135,  13,   8, 186, 137, 129, 246, 433],
                [220, 173, 301, 215, 168, 226, 107, 126, 152, 303],
                [448, 154, 220, 190, 131, 376, 438, 487, 464,  20],
                [ 23, 160, 264,  12, 467,  82, 125, 237, 463, 284],
                [ 39, 241, 214, 191,  85, 306, 181,   2, 321, 241],
                [ 94,  99, 310, 218, 107,  89, 280, 395, 207, 487],
                [101, 175, 134,  63, 192, 187, 371, 103,  27, 144],
                [251, 423, 434, 468, 141, 315, 142,   8, 415, 107],
                [204, 425, 421, 294, 262, 217,  85, 212, 272,  58]],
                dtype=int32)
```

```
In [17]: print(np.min(arr))
```

2

```
In [18]: print(np.max(arr))
```

487

```
In [19]: print(np.mean(arr))
```

222.22

```
In [20]: print(f'{np.std(arr):.2f}')
```

136.50

- Broadcast a 1D array over a 2D array [all possible scenarios]

```
In [21]: arr1 = np.array([5,3,4,2,1])
arr1
```

```
Out[21]: array([5, 3, 4, 2, 1])
```

```
In [22]: arr2 = np.random.randint(1,10,5).reshape(5,1)
arr2
```

```
Out[22]: array([[4],
               [2],
               [9],
               [5],
               [2]], dtype=int32)
```

```
In [23]: arr1 + arr2
```

```
Out[23]: array([[ 9,  7,  8,  6,  5],
               [ 7,  5,  6,  4,  3],
               [14, 12, 13, 11, 10],
               [10,  8,  9,  7,  6],
               [ 7,  5,  6,  4,  3]])
```

```
In [24]: arr1 * arr2
```

```
Out[24]: array([[20, 12, 16,  8,  4],
               [10,  6,  8,  4,  2],
               [45, 27, 36, 18,  9],
               [25, 15, 20, 10,  5],
               [10,  6,  8,  4,  2]])
```

Set 3:

- Find common elements between two arrays [with and without using intersect1d, use isin() instead]
- Subtract row mean from each row of a 2D array
- Given a 4×4 array, swap the first half rows with the second half rows
- Given a 1D array of random numbers, find the indices of the 5 largest values (no sorting/ check argsort() instead).
- From a 2D NumPy array, extract only the unique rows (no duplicates hint: check unique method).

```
In [25]: arr1 = np.random.randint(1,15,4).reshape(2,2)
arr1
```

```
Out[25]: array([[14,  9],
               [13,  8]], dtype=int32)
```

```
In [26]: arr2 = np.random.randint(1,10,4).reshape(2,2)
arr2
```

```
Out[26]: array([[9,  5],
               [3,  9]], dtype=int32)
```

- Find common elements between two arrays [with and without using intersect1d, use isin() instead]

```
In [27]: common_ele = np.isin(arr1,arr2)
print('Common Element : ',arr1[common_ele])
```

```
Common Element :  [9]
```

```
In [28]: common_ele = np.intersect1d(arr1,arr2)
print('Common Element : ',common_ele)
```

```
Common Element :  [9]
```

- Subtract row mean from each row of a 2D array

```
In [29]: arr = np.random.randint(1,10,4).reshape(2,2)
arr
```

```
Out[29]: array([[7,  7],
               [8,  6]], dtype=int32)
```

```
In [30]: mean_val = np.mean(arr,axis=1,keepdims = True)
mean_val
```

```
Out[30]: array([[7.],
               [7.]])
```

```
In [31]: arr - mean_val
```

```
Out[31]: array([[ 0.,  0.],
               [ 1., -1.]])
```

- Given a 4×4 array, swap the first half rows with the second half rows

```
In [32]: arr = np.random.randint(1,25,16).reshape(4,4,)
arr
```

```
Out[32]: array([[ 3, 18, 21,  4],
                [11,  5,  7, 22],
                [12, 23, 15, 14],
                [12, 23, 10,  7]], dtype=int32)
```

```
In [33]: half = np.size(arr[0])/2
swaped = np.vstack((arr[half:],arr[:half]))
swaped
```

```
Out[33]: array([[12, 23, 15, 14],
                [12, 23, 10,  7],
                [ 3, 18, 21,  4],
                [11,  5,  7, 22]], dtype=int32)
```

- Given a 1D array of random numbers, find the indices of the 5 largest values (no sorting/ check argsort() instead)

```
In [34]: arr = np.random.randint(1,50,10)
arr
```

```
Out[34]: array([18, 42, 20, 45,  1,  6,  5, 12,  9, 28], dtype=int32)
```

```
In [35]: print('Indices Of 5 Largest Numbers : ',np.argsort(arr)[np.size(arr)-5:])
```

```
Indices Of 5 Largest Numbers :  [0 2 9 1 3]
```

- From a 2D NumPy array, extract only the unique rows (no duplicates hint: check unique method)

```
In [36]: arr = np.array([[1,2,3],[4,5,6],[1,2,3]])
arr
```

```
Out[36]: array([[1, 2, 3],
                [4, 5, 6],
                [1, 2, 3]])
```

```
In [37]: unique_rows, counts = np.unique(arr, axis=0, return_counts=True)
         unique_only = unique_rows[counts == 1]
         print('Unique Rows : ',unique_only)
```

```
Unique Rows :  [[4 5 6]]
```

```
In [ ]:
```

Exported with [runcell](https://www.runcell.dev) — convert notebooks to HTML or PDF anytime at [runcell.dev](https://www.runcell.dev).